

PROJECT ROADMAP

LedgerStream

A Real-Time, Exactly-Once Payment Ledger
Built on Apache Kafka

Six-day build plan — from infrastructure to
fraud-aware
dashboard — designed for fintech / backend
interview prep

1. Project Overview

WHAT YOU ARE BUILDING

A real-time payment ledger system where money moves between accounts via Kafka events, with a guarantee of **exactly-once processing** — even when consumers crash, messages are redelivered, or failures happen mid-transaction. A secondary fraud-scoring consumer rides on the same event stream. A live dashboard shows balances, transaction flow, and fraud alerts.

The Core Problem You Are Solving

In a naive event-driven pipeline, if a consumer processes a "transfer ₹500 from A to B" event, updates the database, but crashes *before* committing its Kafka offset, Kafka will redeliver that same message. A naive consumer would apply the transfer twice — B ends up with ₹1000 instead of ₹500. LedgerStream demonstrates how to prevent this using idempotency keys, transactional writes, and careful offset management — the exact infrastructure problem every real payment system (UPI, PhonePe, Razorpay, JPMC) has to solve.

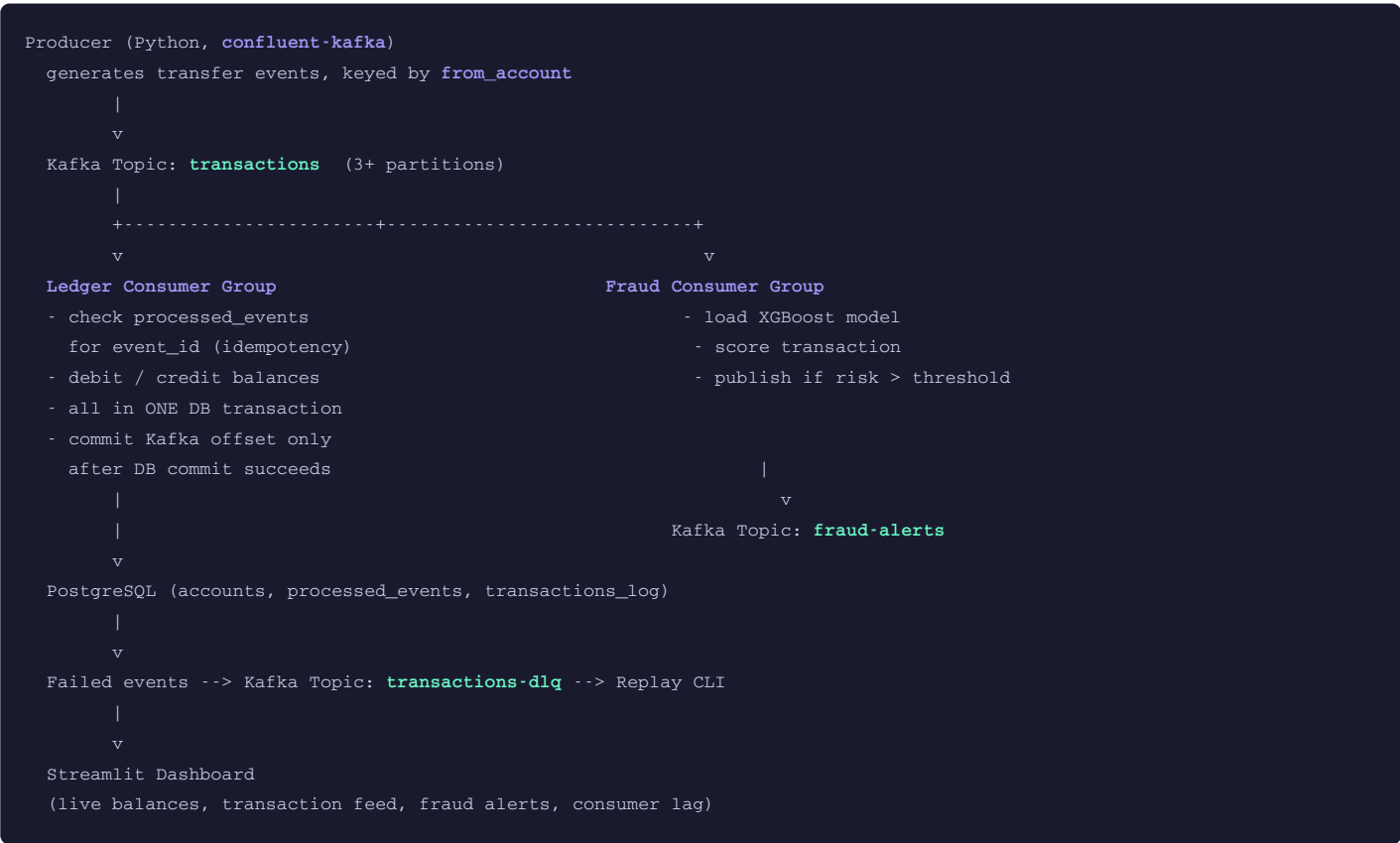
THE INTERVIEW STORY

"I built a Kafka-based ledger system simulating account-to-account transfers. The hard part wasn't producing or consuming events — it was guaranteeing exactly-once semantics. I used a transactional consumer combined with idempotency keys stored in Postgres, so that even if a consumer crashes and Kafka redelivers a message, the transfer is never double-applied. I partitioned by account ID to preserve per-account ordering, since a debit-then-credit sequence must never be processed out of order. I also built a dead-letter queue with a replay tool, and a secondary consumer that scores transactions for fraud on the same stream. I load-tested the pipeline and can speak to what changes at production scale."

Why This Stands Out

Most student Kafka projects stop at "producer sends data, consumer processes it, here's a dashboard." LedgerStream goes further by demonstrating the failure modes that actually matter in production: redelivery, ordering, consistency, and recovery. This is precisely the depth that fintech backend and infrastructure interviews probe for.

High-Level Architecture



2. Complete Tech Stack

Layer	Tool	Purpose
Broker	Apache Kafka (KRaft mode)	Event backbone; no Zookeeper dependency needed
Producer	Python — confluent-kafka	Generates transfer events, partitioned by account ID
Core Consumer	Python, transactional consumer-producer pattern	Applies ledger updates with exactly-once semantics
Idempotency Store	PostgreSQL (unique constraint on event_id)	Prevents double-processing on message redelivery
Fraud Consumer	Python + XGBoost (secondary consumer group)	Scores each transaction, publishes high-risk alerts
DLQ + Replay	Kafka topic + small CLI tool	Captures and recovers from failed events
Dashboard	Streamlit	Live balances, transaction feed, fraud alerts
Orchestration	Docker Compose	Single-command spin-up of the full stack
Load Testing	Custom script + Kafka consumer lag metrics	Honest, benchmarked throughput numbers

3. Day-by-Day Build Roadmap

- 1 Infrastructure & Ledger Schema
- 2 Producer with Account-Based Partitioning
- 3 Exactly-Once Ledger Consumer (the core of the project)
- 4 Dead Letter Queue & Replay Tool
- 5 Fraud Scoring Consumer (secondary stream)
- 6 Dashboard, Load Test & Production Writeup

DAY 1 — Infrastructure + Ledger Schema

Set up the foundational infrastructure and design the database that will hold your ledger state.

- ☐ Write a `docker-compose.yml` with Kafka running in KRaft mode (no Zookeeper) and a PostgreSQL service.
- ☐ Create the `transactions` Kafka topic with at least 3 partitions.
- ☐ Design the Postgres schema: `accounts` (`account_id`, `balance`), `processed_events` (`event_id` UNIQUE, `status`, `processed_at`), `transactions_log` (full audit trail of every transfer).
- ☐ Seed the `accounts` table with a handful of test accounts and starting balances.

END OF DAY 1

Kafka and Postgres running via one `docker-compose up`, with the schema created and seed data loaded.

DAY 2 — Producer with Partitioning

Build the event producer and make the key architectural decision that protects ordering.

- ☐ Write a Python producer that generates transfer events: `event_id`, `from_account`, `to_account`, `amount`, `timestamp`.
- ☐ Key every message by `from_account` so all events for a given account land on the same partition — this preserves per-account ordering.
- ☐ Add a configurable send rate (e.g. one event every 50-200ms) to simulate realistic traffic.
- ☐ Manually inspect partition assignment to confirm the same account always lands on the same partition.

END OF DAY 2

A running producer that streams transfer events into Kafka with verified, deterministic partitioning by account.

DAY 3 — Exactly-Once Ledger Consumer (Core)

This is the heart of the project. Spend the most time here.

- ☐ Build the ledger consumer: for each message, first check `processed_events` for the `event_id`.
- ☐ If the `event_id` already exists — skip it. This is your idempotency guard.
- ☐ If it does not exist — in a **single database transaction**, debit the sender, credit the receiver, insert into `processed_events`, and write to `transactions_log`.
- ☐ Only commit the Kafka consumer offset *after* the database transaction has successfully committed.
- ☐ Deliberately kill the consumer mid-processing and restart it to prove that redelivered messages do not get double-applied.

COMMON PITFALL TO AVOID

Committing the Kafka offset before the database write completes. If you do this and the process crashes between the offset commit and the DB write, the event is lost forever — the opposite failure mode, and just as bad as double-processing.

END OF DAY 3

A consumer that survives crash-and-redeliver tests without ever double-processing or losing a transfer.

DAY 4 — Dead Letter Queue + Replay

Handle the messages that fail to process cleanly, the way real on-call engineers do.

- ☐ Wrap the consumer's processing logic in `try/except`.
- ☐ On failure (malformed event, DB error, insufficient balance, etc.), publish the original message plus error metadata to a `transactions-dlg` topic instead of crashing or silently dropping it.
- ☐ Build a small CLI tool that lists messages currently sitting in the DLQ.

- ☐ Extend the CLI to let you fix and replay a DLQ message back into the main `transactions` topic.

END OF DAY 4

A working DLQ with a replay tool you can demo: break a message on purpose, watch it land in the DLQ, fix it, and replay it successfully.

DAY 5 — Fraud Scoring Consumer

Add a secondary consumer group that rides on the same event stream for fraud detection.

- ☐ Train an XGBoost model on the Kaggle "Credit Card Fraud Detection" dataset (or similar), handling class imbalance properly (class weighting or SMOTE) rather than relying on raw accuracy.
- ☐ Pickle the trained model and load it inside a new, independent consumer group subscribed to the same `transactions` topic.
- ☐ Score each transaction as it arrives; publish high-risk transactions (score above a chosen threshold) to a `fraud-alerts` topic.
- ☐ Confirm that this consumer group operates independently of the ledger consumer group — both read the same topic without interfering with each other.

END OF DAY 5

A second, independent consumer scoring every transaction for fraud risk in parallel with ledger processing.

DAY 6 — Dashboard, Load Test & Writeup

Make the system visible, measure it honestly, and document what you would do differently at scale.

- ☐ Build a Streamlit dashboard showing: live account balances, a scrolling transaction feed, a high-risk/fraud alert table, and consumer lag.
- ☐ Write a load-testing script that increases the producer's send rate in steps, while you observe consumer lag and end-to-end latency.
- ☐ Record your *actual* measured throughput — do not estimate or guess this number for your resume.
- ☐ Write a short "at production scale" section covering: multi-broker clusters, Schema Registry for Avro/Protobuf, partition rebalancing, and monitoring/alerting infrastructure.

END OF DAY 6

A fully running, dashboarded system with real benchmark numbers and a written reflection on production-scale gaps — ready to demo and discuss in interviews.

4. Key Interview Talking Points

Concept	What To Say
Exactly-once semantics	Idempotency key (event_id) check before processing, combined with atomic DB transaction + offset commit ordering.
Partitioning strategy	Keyed by account ID so per-account event ordering is guaranteed within a partition.
Failure recovery	Dead-letter queue isolates bad events without blocking the rest of the stream; replay tool restores them once fixed.
Consumer groups	Ledger processing and fraud scoring run as two independent consumer groups on the same topic, scaling and failing independently.
Honest benchmarking	Measured throughput and consumer lag from your own load test — not an assumed or copied number.
Production gap awareness	Multi-broker replication, Schema Registry, partition rebalancing, and monitoring are explicitly identified as next steps, not claimed as already solved.